DAY 08 · 01

설비 고장을 예측하는 AI 데이터 분석

모듈·표준 라이브러리·파일 읽기 — 모듈과 경로 다루기

posco

CONTENTS

오늘 다룰 내용

PART 01

모듈과 import

코드를 가져다 쓰는 방법 · import 4가지 방식

PART 02

표준 라이브러리

math · random · datetime · os · csv 다섯 모듈 살펴보기

PART 03

경로와 os 모듈

절대·상대경로, getcwd · listdir · path.exists · path.join

posco

앞으로 다룰 파일이 어떤 설비 데이터인지 먼저 이해

공정 — 소성가공 프레스

금속을 강한 힘으로 눌러 형태를 만드는 설비. 작동 중 진동과 전류를 지속적으로 기록한다.

정상과 이상

정상이면 진동·전류가 일정 범위에 머문다. 금형 마모나 이물이 끼면 값이 튀고 상태가 1로 기록된다.

다룰 파일

data/08_press.csv — 설비ID·시각·진동X·진동Y·전류·상태. 이번 파일·경로 실습의 대상 실데이터.

같은 코드를 매번 다시 짜는 문제

> **도메인이 자산, AI와 도구는 가져다 쓰는 것**

필요한 기능은 이미 누군가 만들어 두었습니다.
제곱근 계산만 해도 공식을 알아야 하고 정확도를 맞추기도 어렵죠.
다행히 이런 기능들은 잘 만들어져 있고, 우리는 **가져다 쓰기만 하면 됩니다**.

지난 시간까지 변수, 조건문, 반복문, 함수를 배웠고 직접 함수를 만들 수 있게 됐습니다. 하지만 필요한 **모든 것을 직접 만들 필요는 없습니다**.

직접 만들기보다 가져다 쓰기

좋은 개발자는 검증된 도구를 잘 찾아 조합하는 사람

A · BUILD

직접 제작

공식을 이해하고 **정확도까지 확보**해야 합니다.
시간이 많이 들고, 검증 책임도 본인에게 있습니다.

B · USE

가져다 쓰기

검증된 기능을 **import 한 줄**로 사용합니다.
NumPy·Pandas 같은 강력한 도구도 모두 이 방식입니다.

모듈이란 무엇인가

모듈은 코드가 담긴 .py 파일 하나

파이썬 코드가 담긴 파일 하나가 바로 모듈입니다.
망치와 드라이버가 든 도구 상자처럼,
math 모듈은 수학 도구가 담긴 상자,
os 모듈은 폴더·파일 도구가 담긴 상자입니다.

함수와 변수가 모여 있는 **도구 상자**를 가져와 필요한 도구를 꺼내 씁니다.

math.py

```
def sqrt(x): ...
 def ceil(x): ...
 def floor(x): ...
 pi = 3.14159...
 e = 2.71828...
```

모듈을 쓰는 세 가지 이유

재사용 · 관리 · 검증 — 모듈이 주는 세 가지 이득

01

재사용

한 번 만든 코드를 **다른 파일에서도** 사용합니다.
복사·붙여넣기 없이
import 한 줄이면 충분합니다.

관리

길어진 코드를 **기능별 파일로 나눠** 정리합니다.
찾기도 쉽고 고치기도 쉬워집니다.

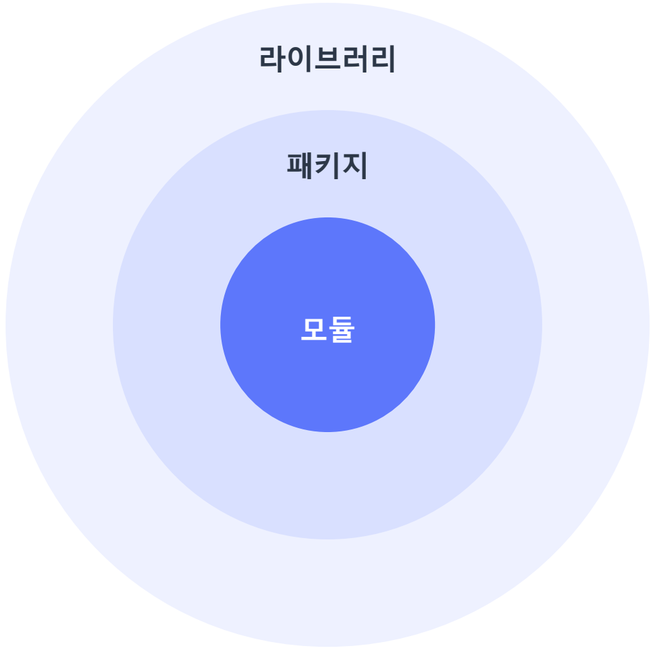
검증

전 세계 개발자가 **검증한 코드**를
그대로 사용합니다.
직접 만드는 것보다 안전합니다.

posco

모듈·패키지·라이브러리 구분

모듈은 파일, 패키지는 폴더, 라이브러리는 통칭



SCOPE 01

모듈 — 파일 하나

코드가 담긴 **.py 파일** 한 개가 단위.

SCOPE 02

패키지 — 폴더

관련 **모듈을 모은 폴더**가 단위.

SCOPE 03

라이브러리 — 통칭

셋 모두 **"가져와 쓴다"**는 공통점, 방법은 import.

import 기본 사용법

import 모듈명 한 줄로 그 모듈의 모든 기능 사용 준비 완료

```
import 모듈명
모듈명.기능명()
```

가져온 뒤에는 모듈명.기능명() 형태로 꺼내 씁니다.

import는 보통 코드 파일 **맨 위에 한 줄씩 모아** 적습니다. 이 코드가 어떤 도구를 쓰는지 한눈에 보이기 때문입니다.

posco

import 기본 — 코드로 확인

math 모듈을 가져와 제곱근 구하기

CODE

```
import math
result = math.sqrt(16)
print(result)   # 출력: 4.0
```

EXPLAIN

한 줄로 도구 상자를 통째로

import math 한 줄로 math 상자를 가져오고,
math.sqrt(16)으로
제곱근 도구를 꺼내 16에 씁니다. 결과는 **4.0**.

※ import 없이 sqrt(16)만 쓰면 오류 — sqrt는 math 안 math 안에 있기 때문입니다.

posco

점 표기법으로 기능 꺼내쓰기

점은 "~의 안에 있는" 이라는 뜻

```
math.sqrt
```

▲ SOURCE
점 앞 = 출처

▲ FUNCTION
점 뒤 = 기능

이 구조만 기억하면 어떤 코드든 읽힙니다. 앞으로 볼 pd.read_csv, np.mean도 모두 같은 원리입니다.

pd는 Pandas, np는 NumPy를 가리키고, 점 뒤 기능을 그 모듈에서 꺼내 씁니다.

from import 로 일부만 가져오기

from 모듈명 import 기능명 — 필요한 기능만 골라오기

```
from 모듈명 import 기능명
기능명()# 모듈명 없이 바로 사용
```

가져온 뒤에는 모듈명 없이 바로 쓸 수 있습니다. 코드는 간결해지지만 출처가 가려지는 단점이 있습니다.

기능을 골고루 쓰면 **import 모듈명**이, 한두 개만 쓰면 **from … import**가 편합니다.

import 방식 비교

두 방식 모두 자주 보게 되니 눈에 익혀 두기

구분	import 모듈명	from … import
작성	`import math`	`from math import sqrt`
사용	`math.sqrt(16)`	`sqrt(16)`
장점	**출처 명확** — 어디서 온 기능인지 한눈에 드러남	**코드 간결** — 매번 모듈명을 붙일 필요 없음
단점	매번 모듈명을 붙여야 함	출처가 가려져 코드를 추적하기 어려움

posco

as 별칭과 import 별표

np · pd · plt — 전 세계가 따르는 공통 약속

✓ RECOMMENDED · 권장

as 별칭

긴 모듈명에 짧은 별명을 붙입니다.
데이터 분석에는 **전 세계가 따르는 공통 약속**이 있습니다.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

✕ AVOID · 실무 회피

import 별표

모든 기능을 한꺼번에 가져옵니다. 내 변수와의 **이름 충돌**과 **출처 가림** 때문에 실무에서는 피합니다.

```
from math import*
# sqrt, pi, ceil, ... 전부 한꺼번에
# → 어디서 왔는지 알 수 없음
```

as 별칭 — 코드로 확인

모듈에 공통 약속 별명 붙이기

CODE

```
import datetime as dt
now = dt.datetime.now()
print(type(now))   # <class 'datetime.datetime'>
```

EXPLAIN

긴 이름엔 별명을

긴 이름 **datetime**에 **dt**라는 별명을 붙였습니다.
그 뒤로는 **dt.datetime.now()**처럼 짧게 씁니다.

※ numpy → np, pandas → pd는 처음부터 약속대로 쓰로 쓰는 습관을 들이세요.

실습 1. import 세 방식으로 모듈 가져오기

목표

import·from import·as 세 방식으로 같은 기능을 가져와 써 보기

단계

① import 모듈명으로 통째로 가져와 모듈명.기능() 으로 사용

② from 모듈 import 기능 으로 일부만 가져와 모듈명 없이 사용

③ import 모듈 as 별명 으로 별명.기능() 으로 사용

④ 세 방식의 출력이 같은지 확인

예상 결과

세 방식 모두 4.0 5 출력

posco

실습 1. import 세 방식으로 모듈 가져오기 정답

CODE · 정답

```
import math
print(math.sqrt(16), math.ceil(4.2))   # 4.0 5
from math import sqrt, ceil
print(sqrt(16), ceil(4.2))             # 4.0 5 (모듈명 없이)
import math as m
print(m.sqrt(16), m.ceil(4.2))         # 4.0 5
```

posco

표준 라이브러리란

파이썬 설치 시 함께 깔리는 검증된 모듈 모음

자주 쓸 도구 상자 상당수는 파이썬을 깔 때 **이미 함께 깔렸습니다**. 따로 설치하지 않아도 **import만 하면 바로** 쓸 수 있죠.

> PYTHON PHILOSOPHY
>
> **Batteries Included — 장난감에 건전지까지 들어 있어 바로 가지고 놀 수 있다는 뜻입니다**

math · random · datetime · os · csv가 모두 여기 속합니다.

자주 쓰는 표준 라이브러리 다섯

데이터 분석 입문에서 자주 만나는 다섯 가지

01

math

수학 계산

제곱근, 올림·내림 등 한 단계 전문적인 계산

02

random

무작위 처리

가짜 센서값 생성, 무작위 선택

datetime

날짜 · 시간

설비 점검 시각 기록

os

폴더 · 파일

현재 위치, 파일 목록, 경로 조립

csv

표 데이터

센서 CSV 읽기·쓰기

표준 라이브러리에는 **수백 개 모듈**이 있지만, 입문 단계에서 만날 건 이 다섯입니다. 지금은 각자 **무슨 일을 하는지 큰 그림**만 잡으면 됩니다.

표준 라이브러리 5종 정리

한자리에 모아 보면 역할이 분명해진다

모듈	하는 일	활용 예
`math`	**수학 계산**	제곱근 · 원주율 — 통계 계산
`random`	**무작위 처리**	가짜 센서값 — 실제 데이터가 없을 때의 연습용
`datetime`	**날짜 · 시간**	점검 시각 — 언제 했는지 기록
`os`	**폴더 · 파일**	파일 목록 — 데이터 폴더 탐색
`csv`	**표 데이터**	센서 CSV — 표 형태 파일 입출력

posco

math 모듈 살펴보기

math는 한 단계 전문적인 계산에 꺼내 쓰는 도구

FUNCTION

`math.sqrt`

제곱근 계산

표준편차 계산 등 통계 작업에 사용합니다.

```
math.sqrt(9) → 3.0
```

FUNCTION

`math.ceil · floor`

올림 · 내림

측정값 정리에 사용합니다.

```
math.ceil(4.2) → 5
math.floor(4.8) → 4
```

BUILT-IN

`abs · round · **`

기본 함수 — math 없이 바로 사용

절댓값, 반올림, 거듭제곱은 import 없이 씁니다.

```
abs(-3) → 3
2 ** 3 → 8
```

posco

math 모듈 — 코드로 확인

제곱근 · 올림 · 내림과 기본 함수 구분

CODE

```
import math
print(math.sqrt(9))     # 출력: 3.0
print(math.ceil(4.2))   # 출력: 5
print(2 ** 3)           # 출력: 8
```

EXPLAIN

전문 계산 vs 기본 계산

sqrt·ceil은 math 상자에서 꺼내 쓰고, **거듭제곱 (**)**·abs·round는 import 없이 바로 씁니다.

※ 기본 계산과 전문 계산을 구분하는 감각을 기르세요.

random 모듈 살펴보기

randint는 양 끝 포함, choice는 무작위 선택

CODE

```
import random
print(random.randint(1, 10))              # 1~10 중 무작위 정수
print(random.choice(['정상', '경고', '위험']))   # 셋 중 무작위 (실행마다 다름)
```

EXPLAIN

양 끝 포함이 핵심

randint(1, 10)은 1 이상 10 이하의 정수 — **양 끝 1과 10도 포함**됩니다.

choice는 리스트에서 하나를 무작위로 고릅니다.

※ 실제 데이터가 없을 때 가짜 센서값을 만들어 연습하기에 좋습니다.

datetime 모듈 살펴보기

현재 시각 가져오기 — 점 두 개에 주의

CODE

```
import datetime
now = datetime.datetime.now()   # 앞 datetime은 모듈, 뒤는 그 안의 도구
print(now)   # 예: 2024-03-01 09:00:00.123456
```

EXPLAIN

점이 두 번 나오는 이유

앞 **datetime**은 모듈 이름,

뒤 **datetime**은 그 안의 도구입니다.

번거로우면

from datetime import datetime으로

짧게 쓸 수 있습니다.

※ 설비 점검은 언제 했는지가 중요하므로 datetime은 꼭 필요합니다.

posco

모듈 문서 읽는 법

다 외울 필요 없음 — 필요할 때 찾는 법만 알면 됨

TOOL 01

dir()

모듈에 어떤 기능이 있는지 **목록을 펼쳐 보여** 줍니다.

```
dir(math)
```

→ ['acos', 'ceil', 'cos', 'e', 'floor', 'pi', 'sqrt', ...]

TOOL 02

help()

특정 기능을 **어떻게 쓰는지 설명서**를 보여 줍니다.

```
help(math.sqrt)
```

→ sqrt(x, /)

Return the square root of x.

표준 vs 외부 라이브러리

"No module named ..." 오류는 대개 외부 라이브러리 미설치

구분	표준 라이브러리	외부 라이브러리
예시	`math` · `os` · `csv` · `random` · `datetime`	`NumPy` · `Pandas` · `Matplotlib`
설치	**불필요** — 파이썬과 함께 설치됨	**pip install**로 따로 설치
포함	파이썬 기본 포함	따로 배포
사용	import로 바로 사용	설치 후 import

실습 2. 표준 라이브러리로 센서값 만들기

목표

random·math 모듈로 무작위 센서값을 만들고 가공하기

단계

① random 모듈을 import

② randint로 무작위 센서값을 만들어 출력

③ math 모듈로 그 값을 가공(제곱근)

④ 다시 실행하면 값이 달라지는지 확인

예상 결과

(매번 다른) 무작위 값과 그 제곱근

posco

실습 2. 표준 라이브러리로 센서값 만들기 정답

CODE · 정답

```
import random
import math
value = random.randint(1, 100)
print(value)             # 매번 다른 값
print(math.sqrt(value))  # 그 값의 제곱근
```

폴더 구조와 경로 표기

슬래시는 "폴더 안으로 들어간다"는 의미

```
documents/
 └ project/
   └ data/
     └ 08_press.csv
```

```
PATH
documents/project/data/08_press.csv
```

폴더는 나무를 거꾸로 세운 계층 구조이고,
경로는 그 가지를 따라가는 길을 적은 것입니다.

※ 윈도우는 역슬래시(\), 맥·리눅스는 슬래시(/) — 이 차이는 os.path.join이 자동으로 맞춰 줍니다.

절대경로와 상대경로

실무는 짧고 옮겨도 동작하는 상대경로를 더 자주 사용

구분	절대경로	상대경로
기준	최상위 폴더부터 끝까지 적음	**현재 위치**에서 출발
예시	`C:\Users\me\project\data\a.csv`	`data/a.csv`
길이	길다 — 우편 주소처럼	짧다 — "여기서 오른쪽으로" 식
이동	**옮기면 오류**	**옮겨도 동작**

※ 점 하나(.) = 현재 폴더 · 점 두 개(..) = 상위 폴더

작업 디렉토리 개념

작업 디렉토리는 코드가 기준으로 삼는 "현재 폴더"

작업 디렉토리(**cwd**, current working directory)는 코드가 "지금 여기 서 있다"고 여기는 폴더입니다. **상대경로는 모두 이 작업 디렉토리에서 출발**합니다.

> **주의**
>
> **작업 디렉토리는 코드 파일 위치와 항상 같지는 않습니다**
>
> 그래서 파일 바로 옆에 데이터를 뒀는데도 못 찾는 일이 생깁니다. **읽기 전에 현재 위치부터 확인**하는 습관이 중요합니다.

os 모듈 소개와 getcwd

os 모듈 — 컴퓨터의 폴더·파일을 다루는 도구

ABOUT

os = Operating System

운영체제와 대화하는 도구입니다. 표준 라이브러리이므로 **import os**만 쓰면 바로 사용할 수 있습니다.

FUNCTION

os.getcwd()

get current working directory — 현재 작업 디렉토리를 **절대경로 문자열**로 돌려줍니다.

데이터 작업의 첫 단계로 **현재 위치를 출력해 보는 것**이 좋은 습관입니다.

posco

getcwd — 코드로 확인

현재 작업 디렉토리를 문자열로 반환

CODE

```
import os
cwd = os.getcwd()
print(cwd)   # 현재 폴더의 절대경로 (예: /home/user/project)
```

EXPLAIN

괄호 안은 비워 둡니다

"현재 위치가 어디냐"고 묻는 것이므로 **괄호 안에는 아무것도 넣지 않습니다**.

※ 출력된 위치가 예상과 다르면, 데이터 파일 경로를 그에 맞게 조정합니다.

os.listdir로 폴더 들여다보기

폴더 안 파일 · 폴더 이름을 리스트로 반환

```
os.listdir(경로) # → list[str]
```

`list directory`의 약자 · 경로 생략 시 현재 작업 디렉토리 대상

결과가 리스트라는 점이 핵심입니다. 지난 시간 배운 **반복문**으로 이 목록을 돌면, 폴더 안 모든 파일을 한 번에 처리할 수 있습니다. 파일이 수십 개여도 코드 몇 줄로 끝납니다

listdir — 코드로 확인

폴더 목록을 받아 반복문으로 출력

CODE

```
import os
files = os.listdir("data")
for name in files:
    print(name)   # 예: 08_sensor_log.txt / 08_press.csv
```

EXPLAIN

파일과 폴더를 함께 출력

listdir은 파일과 폴더를 **구분 없이** 모두 보여 줍니다.

CSV만 고르려면 이름이 **.csv로 끝나는지** 조건문 문으로 확인해야 합니다.

파일 존재 확인의 필요성

읽기 전에 확인하면 프로그램이 멈추지 않음

✕ NO CHECK

바로 열기 → FileNotFoundError

없는 파일을 읽으면 **그 자리에서 멈춥니다**.

파일 100개 중 50번째가 없으면 **나머지 50개도 처리되지 않습니다**.

```
f = open('data/08_없는파일.csv')
FileNotFoundError# 중단
```

✓ CHECK FIRST

존재 확인 → 안전한 처리

읽기 전에 **있는지 확인**하고, 있으면 읽고 **없으면 건너뜁니다**.

택배 기사가 초인종을 누르기 전에 주소를 확인하는 것과 같습니다.

```
if os.path.exists(path):
 f = open(path)
```

os.path.exists 와 os.path.join

os.path — 경로 전문 도구

FUNCTION 01

os.path.exists()

경로에 파일이 **있으면 True, 없으면 False**를 돌려줍니다. 조건문과 함께 썬 있을 때만 읽도록 만듭니다.

```
os.path.exists('data/08_press.csv')
# → True 또는 False
```

FUNCTION 02

os.path.join()

폴더명과 파일명을 합쳐 **완전한 경로**로 만듭니다. **OS에 맞는 구분자**를 자동으로 넣어 주므로 윈도우 ↔ 맥에서도 그대로 동작합니다.

```
os.path.join('data', '08_press.csv')
# → 'data/08_press.csv' (mac/linux)
# → 'data\\08_press.csv' (windows)
```

exists · join — 코드로 확인

안전하게 경로 조립 후 존재 확인

CODE

```
import os
path = os.path.join("data", "08_press.csv")
if os.path.exists(path):
    print('파일 있음:', path)
```

EXPLAIN

join + exists 조합

join으로 폴더명과 파일명을 안전하게 합쳐 경로를 만들고,

exists로 파일 존재를 확인해 있을 때만 처리합니다.

※ 슬래시로 직접 잇지 말고 join을 쓰는 게 OS 독립적이라 안전합니다.

posco

실습 3. os로 폴더 목록 살펴보기

목표

os로 현재 위치를 확인하고 폴더 안 파일 목록을 순회하기

단계

① os 모듈을 import

② getcwd로 현재 작업 폴더를 확인

③ listdir로 폴더 안 목록을 변수에 담기

④ for로 목록을 하나씩 출력하고 csv만 골라 출력

예상 결과

현재 경로 / 폴더 안 파일들 / .csv 파일만

PART 02 · 실습

실습 3. os로 폴더 목록 살펴보기 정답

CODE · 정답

```
import os
print(os.getcwd())          # 현재 작업 폴더
files = os.listdir("data")
for name in files:
    print(name)
for name in files:
    if name.endswith(".csv"):
        print(name)
```

실습 4. os로 파일 존재 확인하기

목표

os.path.join으로 경로를 만들고 exists로 파일 존재를 판단하기

단계

① os를 import

② path.join으로 폴더와 파일 이름을 이어 경로를 만들기

③ path.exists로 그 경로가 있는지 참·거짓 확인

④ if로 있으면·없으면 다른 메시지 출력

예상 결과

True 또는 False / 파일 있음 또는 파일 없음

실습 4. os로 파일 존재 확인하기 정답

CODE · 정답

```
import os
path = os.path.join("data", "08_press.csv")
print(os.path.exists(path))   # True 또는 False
if os.path.exists(path):
    print("파일 있음")
else:
    print("파일 없음")
```

실습 5. datetime으로 점검 기록 남기기

목표

os와 datetime으로 폴더 파일 수와 점검 시각을 한 줄로 기록하기

단계

① os와 datetime을 import

② listdir로 폴더 파일 수를 구하기

③ datetime.now로 현재 시각을 담기

④ f-string으로 파일 수와 시각을 한 문장으로 출력

예상 결과

파일 3개, 점검 시각 2026-... 형식 한 줄

실습 5. datetime으로 점검 기록 남기기 정답

CODE · 정답

```
import os
import datetime
files = os.listdir("data")
now = datetime.datetime.now()
print(f"파일 {len(files)}개, 점검 시각 {now}")
```

실습 6. 폴더에서 csv 파일만 골라내기

목표

os로 폴더에서 csv 파일만 골라 새 리스트로 모으고 경로를 만들기

단계

① os를 import하고 listdir로 폴더 목록을 구하기

② for-if로 .csv로 끝나는 이름만 빈 리스트에 모으기

③ 모은 csv마다 path.join으로 전체 경로를 만들기

④ 골라낸 csv 목록을 출력

예상 결과

[CSV] 목록 (csv 파일만)

posco

실습 6. 폴더에서 csv 파일만 골라내기 정답

CODE · 정답

```
import os
files = os.listdir("data")
csvs = []
for n in files:
    if n.endswith(".csv"):
        csvs.append(n)
print("[CSV]")
for n in csvs:
    path = os.path.join("data", n)
    print(" -", path)
```